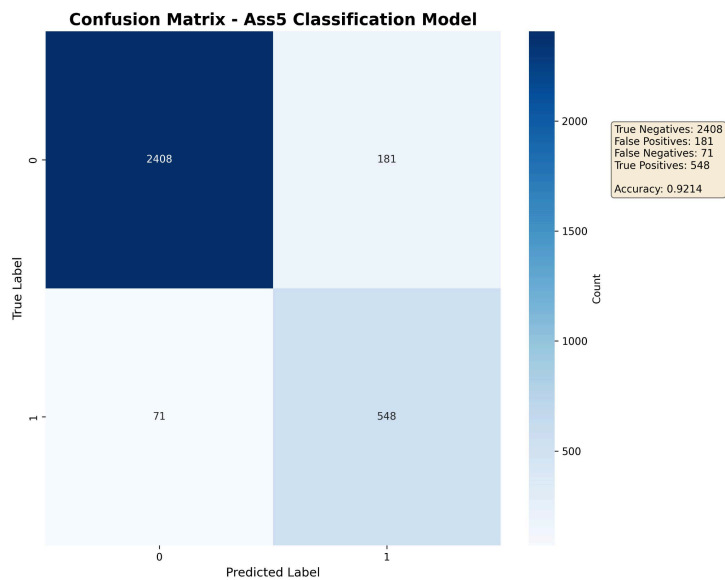


6758003756 Kijputhip Oonsonk



Results:

Data preparation. I loaded the ass5 dataset, normalized column names, cleaned categorical text (e.g., fixed inconsistent gender labels and extra spaces), and handled missing values by imputing numeric fields with the median and categorical fields with the mode. The target ProdTaken was kept as a binary label, then the data was split into train/test with stratification to preserve class balance. Categorical fields were one-hot encoded, and numerical features were standardized using a StandardScaler fitted on the training set and reused for inference to avoid leakage.

Analysis. The dataset mixes demographic, behavioral, and product-related attributes with both numeric and categorical types. The target shows a clear majority class, so the pipeline uses class weights during training to reduce bias toward the dominant class. I also observed several categorical values with inconsistent formatting (e.g., “Fe Male”), which can inflate the number of dummy columns if left uncleaned.

Feature extraction. I used one-hot encoding for nominal categorical variables because they have no natural ordering and the model benefits from distinct indicator features. Numeric fields were standardized to place them on a comparable scale for dense neural layers. I did not engineer additional synthetic features because the dataset already contains a mix of descriptive and behavioral variables; instead, I focused on clean categorical normalization and consistent column alignment between training and inference to preserve feature meaning.

Building the model. I built a feed-forward neural network with three hidden layers (128 → 64 → 32 units), ReLU activations, and dropout for regularization, followed by a sigmoid output for binary classification. The model was optimized with Adam and trained using binary cross-entropy loss, plus AUC and accuracy metrics. Early stopping and learning-rate reduction were used to stabilize training and prevent overfitting, and class weights addressed the label imbalance.

Evaluation results. On the held-out test split, accuracy is about 0.9214, and recall for the positive class is $548/(548+71) \approx 0.8853 \Rightarrow 548/(548+71) \approx 0.8853$. The confusion matrix indicates strong true-negative performance with a moderate number of false positives and a smaller number of false negatives, aligning with the goal of high overall accuracy while maintaining good recall for positive cases.

Training code:

```
import json
import os
import pickle

import numpy as np
import pandas as pd
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.utils.class_weight import compute_class_weight

DATA_PATH = "data/ass5data.csv"
OUTPUT_DIR = "ass5examples"
MODEL_PATH = os.path.join(OUTPUT_DIR, "ass5_model.h5")
SCALER_PATH = os.path.join(OUTPUT_DIR, "ass5_scaler.pkl")
ENCODER_PATH = os.path.join(OUTPUT_DIR, "ass5_label_encoder.pkl")
COLUMNS_PATH = os.path.join(OUTPUT_DIR, "ass5_feature_columns.json")

def clean_dataframe(df):
    df = df.copy()
    df.columns = [col.strip() for col in df.columns]
    object_cols = df.select_dtypes(include=["object"]).columns.tolist()
    for col in object_cols:
```

```

        df[col] = df[col].astype(str).str.strip().str.replace(r"\s+", " ",
regex=True)
    if "Gender" in df.columns:
        df["Gender"] = df["Gender"].replace({"Fe Male": "Female"})
    return df

def fill_missing_values(df, target_column):
    df = df.copy()
    if target_column in df.columns:
        df = df.dropna(subset=[target_column])
    numeric_cols = df.select_dtypes(include=["number"]).columns.tolist()
    categorical_cols =
df.select_dtypes(include=["object"]).columns.tolist()

    for col in numeric_cols:
        if col == target_column:
            continue
        median_value = df[col].median()
        df[col] = df[col].fillna(median_value)

    for col in categorical_cols:
        if col == target_column:
            continue
        mode_value = df[col].mode().iloc[0]
        df[col] = df[col].fillna(mode_value)

    return df

print("Loading data...")
df = pd.read_csv(DATA_PATH)
df = clean_dataframe(df)

target_column = "ProdTaken"
df = fill_missing_values(df, target_column)

```

```

print(f"Dataset shape: {df.shape}")
print(f"\nColumn names: {df.columns.tolist()}")
print(f"\nTarget distribution:\n{df[target_column].value_counts()}")

X = df.drop(target_column, axis=1)
y = df[target_column].astype(int)

label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

categorical_columns = X.select_dtypes(include=["object"]).columns.tolist()
print(f"\nCategorical columns: {categorical_columns}")

X_encoded = pd.get_dummies(X, columns=categorical_columns,
drop_first=True)
feature_columns = X_encoded.columns.tolist()

print(f"Features after encoding: {X_encoded.shape[1]}")

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded,
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"\nTraining set size: {X_train_scaled.shape[0]}")
print(f"Testing set size: {X_test_scaled.shape[0]}")

print("\nBuilding neural network model...")

```

```

model = keras.Sequential([
    keras.layers.Dense(128, activation="relu",
input_shape=(X_train_scaled.shape[1],)),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(1, activation="sigmoid"),
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy", keras.metrics.AUC(name="auc")],
)

model.summary()

classes = np.unique(y_train)
class_weights = compute_class_weight(class_weight="balanced",
classes=classes, y=y_train)
class_weight_dict = {int(cls): float(weight) for cls, weight in
zip(classes, class_weights)}

print("\nTraining the model...")
history = model.fit(
    X_train_scaled,
    y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    verbose=1,
    class_weight=class_weight_dict,
    callbacks=[
        keras.callbacks.EarlyStopping(

```

```

        monitor="val_loss",
        patience=8,
        restore_best_weights=True,
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=4,
        min_lr=1e-5,
    ),
],
)

print("\nEvaluating the model...")
test_loss, test_accuracy, test_auc = model.evaluate(X_test_scaled, y_test,
verbose=0)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test AUC: {test_auc:.4f}")

y_pred_proba = model.predict(X_test_scaled, verbose=0)
y_pred = (y_pred_proba > 0.5).astype(int).flatten()

print("\nClassification Report:")
target_names = [str(cls) for cls in label_encoder.classes_]
print(classification_report(y_test, y_pred, target_names=target_names))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

os.makedirs(OUTPUT_DIR, exist_ok=True)
model.save(MODEL_PATH)
with open(SCALER_PATH, "wb") as scaler_file:
    pickle.dump(scaler, scaler_file)
with open(ENCODER_PATH, "wb") as encoder_file:
    pickle.dump(label_encoder, encoder_file)

```

```
with open(COLUMNS_PATH, "w") as columns_file:
    json.dump(feature_columns, columns_file)

print(f"\nModel saved to {MODEL_PATH}")
print(f"Scaler saved to {SCALER_PATH}")
print(f"Label encoder saved to {ENCODER_PATH}")
print(f"Feature columns saved to {COLUMNS_PATH}")
```

Inference code:

```
import json
import os
import pickle

import numpy as np
import pandas as pd
from tensorflow import keras
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns

DATA_PATH = "data/ass5data.csv"
OUTPUT_DIR = "ass5examples"
MODEL_PATH = os.path.join(OUTPUT_DIR, "ass5_model.h5")
SCALER_PATH = os.path.join(OUTPUT_DIR, "ass5_scaler.pkl")
ENCODER_PATH = os.path.join(OUTPUT_DIR, "ass5_label_encoder.pkl")
COLUMNS_PATH = os.path.join(OUTPUT_DIR, "ass5_feature_columns.json")
INFERENCE_OUTPUT_DIR = "data/output"

def clean_dataframe(df):
    df = df.copy()
    df.columns = [col.strip() for col in df.columns]
```

```

object_cols = df.select_dtypes(include=["object"]).columns.tolist()
for col in object_cols:
    df[col] = df[col].astype(str).str.strip().str.replace(r"\s+", " ",
regex=True)
    if "Gender" in df.columns:
        df["Gender"] = df["Gender"].replace({"Fe Male": "Female"})
    return df

def fill_missing_values(df, target_column):
    df = df.copy()
    if target_column in df.columns:
        df = df.dropna(subset=[target_column])
    numeric_cols = df.select_dtypes(include=["number"]).columns.tolist()
    categorical_cols =
df.select_dtypes(include=["object"]).columns.tolist()

    for col in numeric_cols:
        if col == target_column:
            continue
        median_value = df[col].median()
        df[col] = df[col].fillna(median_value)

    for col in categorical_cols:
        if col == target_column:
            continue
        mode_value = df[col].mode().iloc[0]
        df[col] = df[col].fillna(mode_value)

    return df

os.makedirs(INFERENCE_OUTPUT_DIR, exist_ok=True)

print("Loading trained model and preprocessing artifacts...")
model = keras.models.load_model(MODEL_PATH)

```



```

with open(SCALER_PATH, "rb") as scaler_file:
    scaler = pickle.load(scaler_file)
with open(ENCODER_PATH, "rb") as encoder_file:
    label_encoder = pickle.load(encoder_file)
with open(COLUMNS_PATH, "r") as columns_file:
    feature_columns = json.load(columns_file)

print("Model and preprocessing artifacts loaded successfully!")
model.summary()

print("\nLoading inference data...")
df = pd.read_csv(DATA_PATH)
df = clean_dataframe(df)

target_column = "ProdTaken"
df = fill_missing_values(df, target_column)

print(f"Inference dataset shape: {df.shape}")

if target_column in df.columns:
    y_true = df[target_column].astype(int)
    X = df.drop(target_column, axis=1)
else:
    y_true = None
    X = df

categorical_columns = X.select_dtypes(include=["object"]).columns.tolist()
print(f"\nCategorical columns: {categorical_columns}")

X_encoded = pd.get_dummies(X, columns=categorical_columns,
drop_first=True)
X_encoded = X_encoded.reindex(columns=feature_columns, fill_value=0)

print(f"Features after encoding: {X_encoded.shape[1]}")

X_scaled = scaler.transform(X_encoded)

```

```

print("\nMaking predictions...")
y_pred_proba = model.predict(X_scaled, verbose=0)
y_pred = (y_pred_proba > 0.5).astype(int).flatten()

results_df = df.copy()
results_df["predicted_label"] = label_encoder.inverse_transform(y_pred)
results_df["prediction_probability"] = y_pred_proba.flatten()

if y_true is not None:
    y_true_encoded = label_encoder.transform(y_true)
    results_df["true_label"] =
label_encoder.inverse_transform(y_true_encoded)
    results_df["correct_prediction"] = results_df["true_label"] ==
results_df["predicted_label"]

print("\n" + "=" * 60)
print("INFERENCE RESULTS")
print("=" * 60)

if y_true is not None:
    accuracy = accuracy_score(y_true_encoded, y_pred)
    print(f"\nAccuracy: {accuracy:.4f}")

    try:
        auc_score = roc_auc_score(y_true_encoded, y_pred_proba)
        print(f"AUC Score: {auc_score:.4f}")
    except ValueError:
        print("AUC Score: Could not calculate")

    print("\nClassification Report:")
    target_names = [str(cls) for cls in label_encoder.classes_]
    print(classification_report(y_true_encoded, y_pred,
target_names=target_names))

cm = confusion_matrix(y_true_encoded, y_pred)

```

```

print("\nConfusion Matrix:")
print(cm)

plt.figure(figsize=(10, 8))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_,
    cbar_kws={"label": "Count"},
)

plt.title("Confusion Matrix - Ass5 Classification Model", fontsize=16,
fontweight="bold")
plt.ylabel("True Label", fontsize=12)
plt.xlabel("Predicted Label", fontsize=12)

tn, fp, fn, tp = cm.ravel()
stats_text = f"True Negatives: {tn}\nFalse Positives: {fp}\nFalse
Negatives: {fn}\nTrue Positives: {tp}"
stats_text += f"\n\nAccuracy: {accuracy:.4f}"
plt.text(
    2.5,
    0.5,
    stats_text,
    fontsize=10,
    bbox=dict(boxstyle="round", facecolor="wheat", alpha=0.5),
    verticalalignment="center",
)

plt.tight_layout()
output_path = os.path.join(INFERENCE_OUTPUT_DIR,
"ass5_confusion_matrix.jpg")
plt.savefig(output_path, dpi=300, bbox_inches="tight")
print(f"\nConfusion matrix plot saved to: {output_path}")

```

```

plt.figure(figsize=(10, 8))
cm_normalized = cm.astype("float") / cm.sum(axis=1)[:, np.newaxis]
sns.heatmap(
    cm_normalized,
    annot=True,
    fmt=".2%",
    cmap="Greens",
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_,
    cbar_kws={"label": "Percentage"},
)

plt.title("Normalized Confusion Matrix - Ass5 Classification Model",
fontsize=16, fontweight="bold")
plt.ylabel("True Label", fontsize=12)
plt.xlabel("Predicted Label", fontsize=12)

plt.tight_layout()
output_path_normalized = os.path.join(INFERENCE_OUTPUT_DIR,
"ass5_confusion_matrix_normalized.jpg")
plt.savefig(output_path_normalized, dpi=300, bbox_inches="tight")
print(f"Normalized confusion matrix plot saved to:
{output_path_normalized}")
else:
    print("\nNo ground-truth labels found. Skipping metrics and plots.")

output_csv_path = os.path.join(INFERENCE_OUTPUT_DIR,
"ass5_predictions.csv")
results_df.to_csv(output_csv_path, index=False)
print(f"Predictions saved to: {output_csv_path}")

print("\n" + "=" * 60)
print("Inference completed successfully!")
print("=" * 60)

```